

Design Document: Moments Notice Booking Assistant

Shawn Canady | CSC-128 Capstone

Where each required topic is covered

Required topic	Section
1. Who the bot is for and what problem it solves	1. Purpose and audience
2. Intents and entities, and why those	3. Intents, entities, and state
3. Deterministic code vs the model, and the reason for each	4. Where deterministic code was chosen over the model
4. Three failures found in testing and what was done	6. Testing: three failures and what was done
5. One thing it refuses and one thing it hands to a human	7. Refusal, hand-off, privacy, and disclosure
6. What I would build next with another four weeks	9. What I would build next (four more weeks)

1. Purpose and audience

The audience is prospective clients (couples, event planners, corporate organizers) deciding whether and how to book the band Moments Notice. Their questions are repetitive and factual: how much, how far in advance, what's included, what happens if I cancel.

The problem it solves: the band gets the same booking questions over and over, and visitors want answers at any hour. A bot can answer them around the clock, and the band keeps control of what is promised because it can only repeat what is in `knowledge.py`. Anything the band has to decide itself (custom quotes, date confirmation, contracts) is passed to the band instead of answered.

2. Architecture

- `app.py`: Streamlit interface only; keeps the conversation state in the session.
- `dialog.py`: the conversation: routes each message by intent, fills slots, remembers state across turns, and does the travel quote and booking check.
- `intents.py`: classifies a message into one of four intents.
- `entities.py`: extracts miles, event type, and date from free text.
- `grounded_bot.py`: the `faq` path: retrieve, refuse or call the model, convert failures to messages.
- `retriever.py`: TF-IDF search over the knowledge chunks.
- `knowledge.py`: nine chunks, each with an id and source label.

Flow for one message: validate the text; if a slot-filling flow is open, read the message as an answer to it; otherwise classify the intent. `handoff` returns a fixed message, `travel_quote` and `booking_check` start a slot-filling flow, and `faq` retrieves chunks above the threshold (none means the fixed refusal; otherwise the model phrases an answer from the chunks under a prompt that forbids outside knowledge). The reply is returned with its source labels.

The interface and the logic are separate so the logic can be tested with a fake client (`test_bot.py`) without a browser or API key.

3. Intents, entities, and state

Intents

Intent	Slots it collects	Result	Model?
faq	none	Grounded answer or refusal	Yes, only after retrieval finds a chunk
travel_quote	miles	Fee computed from the travel policy, arithmetic shown	No
booking_check	event_type, event_date	Advance-booking guidance for that date	No
handoff	none	Fixed message pointing to the band	No

Why these four intents. Visitor questions fall into four kinds, and each needs a different kind of answer with a different guarantee. A fact question (`faq`) needs an answer that comes only from the band's own text. A price question (`travel_quote`) depends on a number the visitor must supply, so it needs a question back and then exact arithmetic. A date question (`booking_check`) depends on the kind of event and the date, so it needs a question back and then a date comparison. Anything the band has to decide (custom quotes, confirming a date, contracts) must never be answered by the bot, so it gets its own fixed `handoff` path. I stopped at four because each one has its own handler; a new intent is only worth adding if it needs a different handler or a different guarantee.

Entities

Entity	Used by	Why it is extracted
miles	travel_quote	The only input to the travel fee. A wrong distance gives a wrong price.
event_type	booking_check	Decides which advance-booking guidance applies (for example popular spring and fall wedding dates).
event_date	booking_check	Compared against three months out. The comparison needs an exact date.

These are the only three values the bot has to read exactly, so they are the only entities. Regular expressions in `entities.py` read them because a date or a distance must be read exactly or not at all; "no date found" is a better result than a plausible guess. Everything else a visitor asks is handled by retrieval, not by extraction.

State

State lives in `DialogState`: the flow in progress, the slots filled so far, the slot being asked for, how many unreadable answers in a row, and the last `faq` question. That is what makes four things work:

- A visitor can give details in any order or all at once.
- They can interrupt with a side question, which is answered and then followed by a reminder of the open question, with the slots kept.

- They can say "never mind".
- A short follow-up such as "And when do I pay it?" borrows the last topic for retrieval, because on its own it retrieves nothing.

After two unreadable answers in a row the flow is dropped and the visitor is sent to the band.

4. Where deterministic code was chosen over the model

Decision	Why code, not the model
Refusing when nothing is retrieved	A model given an empty context may improvise despite instructions. Code returns the refusal string and never calls the model, so the outcome can't vary.
Choosing what is relevant (TF-IDF + threshold)	Retrieval is a measurable, repeatable step. I set the threshold from printed scores (0.122 to 0.324 for real questions, 0.0 for unrelated). I can unit-test it exactly, which I cannot do with model output.
Error messages	Fixed strings per failure type, so a rate limit always says the same actionable thing and never leaks an exception.
Input limits (empty, over 500 characters)	A simple length check is cheaper and safer than asking a model to cope with bad input.
Source labels	Come from the retrieved chunks' metadata, not from the model, so the model cannot invent a citation.
Intent routing	Pattern rules in <code>intents.py</code> pick the intent, so the same message always takes the same path and the rules can be tested exactly. A model classifier could drift, and a wrong route sends a visitor to the wrong flow.
Entity extraction	Regular expressions read miles, event type, and dates. A date or a distance must be read exactly or not at all.
The travel fee	Arithmetic in <code>dia_log.py</code> : miles past 25, times 2 for the round trip, times 2 dollars. A price quoted by a model could differ between two asks of the same question.
The booking-date check	A date comparison against three months out. Models are unreliable at date arithmetic.
Hand-off	A fixed message. The things the band must decide (custom quotes, specific dates, contracts) should never be answered by the model, however the question is worded.

Where the model is used: only for the part code can't do well, phrasing a short, natural answer from the retrieved text in the `faq` path. It never supplies the facts, the numbers, or the citation.

5. Grounding and refusal

The prompt tells the model to use only the reference text, to answer with the exact refusal sentence when the answer isn't there, to avoid guessing numbers, and to stay under three sentences. Because retrieval can return a related but non-answering chunk (for example, "Can I pay the deposit by credit card?" retrieves the deposit policy, which doesn't mention payment methods), the prompt, not retrieval, must hold in those cases. Section 6 records how that was tested.

6. Testing: three failures and what was done

Failure 1: casual phrasing retrieved the wrong chunk

- **What happened:** Questions such as "yo how much down payment to lock in the band" and "how long you playin" did not reach the deposit and set-length chunks in early `test_phrasing.py` runs.
- **Cause:** Retrieval. TF-IDF matches words, and the chunks only used formal vocabulary such as "deposit" and "performance length".
- **What I did:** I added everyday synonyms ("down payment", "out of town") to the chunks. `test_phrasing.py` now requires formal, casual, and dialect phrasings of the same question to reach the same chunk. Status: fixed for the phrasings I tested.

Failure 2: the deposit answer added a detail the source doesn't state

- **What happened:** "How much is the deposit?" returned "The deposit is 30 percent of the total booking fee." The reference text says a 30 percent deposit is required but never says what it is a percent of.
- **Cause:** The prompt, working on an ambiguous chunk. The source caption named the right chunk (deposit_policy), so retrieval was correct and the model added a detail.
- **What I did:** I added a prompt rule against saying what a number applies to. The retest dropped "total" but still returned "30 percent of the booking fee", so the prompt alone did not hold. A prompt cannot reliably stop a model from filling an obvious gap. The remaining fix is in the data: state in `knowledge.py` what the 30 percent is calculated on, then retest. **Status: still open.**

Failure 3: "out of town, extra cost" reached the cancellation policy

- **What happened:** "What if my party is out of town, extra cost?" reaches the cancellation chunk instead of the travel fee chunk.
- **Cause:** Retrieval. The wording shares few words with the travel chunk, and "party" and "cost" pull toward other chunks.
- **What I did:** No fix in the chunks yet. Two things were done: the source caption makes it diagnosable (a visitor sees the wrong chunk named, which distinguishes a retrieval miss from a model error), and it is recorded as a known limit in `README.md`. The real fix is the embedding retriever in Section 9. **Status: known limit.**

Automated tests

- `test_retriever.py` (10/10): retrieval and the threshold.
- `test_bot.py` (13/13): rate limit, connection error, timeout, bad key, server error, unexpected exception, blank answer, empty and overlong input, refusal without a model call, refusals carrying no source, and the happy path.
- `test_entities.py` (33/33): distance, event type, and date extraction with a fixed "today".
- `test_intents.py` (47/47): the four intents and, as important, that every question the bot already answers stays a `faq`.
- `test_dialog.py` (42 checks): scripted multi-turn conversations with a fake model that records its calls, proving the deterministic intents never reach the model, that state carries across turns, and that the flows survive interruptions, cancellations, past dates, and a missing API key.
- `test_model_properties.py`: property tests on model output without exact strings (not blank, short, no number absent from the reference text, refusal on out-of-scope input).

- `test_phrasing.py`: checks bias, so formal, casual, and dialect phrasings of one question must reach the same chunk. Scripted transcripts are in `TRANSCRIPT_TESTS.md`.

7. Refusal, hand-off, privacy, and disclosure

- **One thing it refuses:** anything not in its knowledge base, including legal or financial advice and prices not stated. Code returns the fixed refusal sentence, and when retrieval finds nothing the model is never called.
- **One thing it hands to a human:** custom quotes, date availability confirmation, and contracts go to the band. The `handoff` intent catches these and returns a fixed message without calling the model.
- **Disclosure:** the page says visitors are talking to software.
- **Privacy:** the question is sent to Groq; the page says so, asks visitors not to enter personal or payment details, and the app does not save conversations.

8. Limitations

- Retrieval is word-based, so a question phrased with none of the chunk's vocabulary can miss. Casual wording such as "down payment" first retrieved the wrong chunk (fixed with synonyms), and "What if my party is out of town, extra cost?" still reaches the cancellation chunk.
- Memory is limited on purpose: the open flow and its slots, plus the last faq topic. It lasts for one browser session, and a follow-up only resolves if it is short and leans on a pronoun or "and".
- Intent rules are patterns, so an unusual phrasing of a quote or booking request can fall through to faq, which refuses or answers from the knowledge base rather than doing harm.
- The bot has no calendar, so it cannot say a date is open. It gives the advance-booking guidance and hands the confirmation to the band.
- The travel quote reads the policy as the miles past 25, counted round trip, at 2 dollars per mile, and it tells the visitor to confirm the final amount with the band.
- Knowledge is static; changing prices means editing `knowledge.py` and redeploying.

9. What I would build next (four more weeks)

The one thing I would change first: replace the word-based TF-IDF retriever with an embedding (semantic) retriever. Two of my three testing failures, the casual phrasing and the "out of town, extra cost" question, came from word matching, and the synonyms I added are a patch that only covers phrasings I thought of. I would keep the threshold and the code-level refusal, and re-pick the threshold from printed scores. The plan below puts that in week 2, after the open data bug is closed.

Week	Work	How I will know it is done
1	Close the open failure and lock in the fixes. State in <code>knowledge.py</code> what the 30 percent deposit is calculated on, then retest. Add a regression test for each of the three failures from Section 6.	"How much is the deposit?" no longer adds a detail the source doesn't state. All three failure cases are automated tests.

Week	Work	How I will know it is done
2	Embedding retriever. Swap it in behind the same retriever interface. Keep the code-level refusal and source captions. Print scores for real and unrelated questions and re-pick the threshold.	"Out of town, extra cost" reaches the travel fee chunk. <code>test_retriever.py</code> and <code>test_phrasing.py</code> pass, with the synonym patches kept as test cases, not as the fix.
3	Broader phrasing tests. Collect 50 or more questions written by band members and friends, not by me. Run them through intents, entities, and retrieval, and fix what falls through to the wrong path.	A written list of misses with a pass rate before and after. Unusual quote or booking phrasings stop falling through to faq.
4	Band-editable knowledge and final check. Move the nine chunks to a file the band can edit, with a validation test that rejects a malformed chunk, so a price change does not mean editing code. Run a final session with the band using the app. Stretch goal: a read-only calendar so <code>booking_check</code> can say a date looks open, while the band still confirms it.	The band changes one price without my help and the bot repeats the new one. A short written summary of what the band's test found.